

Cloud, Réseaux Virtuels et Cloud Computing

UE CRV — Sorbonne Université

Naceur.Malouch@lip6.fr

Notes de cours – Introduction au Cloud & Virtualisation

Intervenants : Naceur Malouch, Binh-Minh Bui-Xuan, Arthur Escriptou

Mars 2026

Table des matières

1	Introduction au Cloud	3
1.1	Contenu de l'UE CRV	3
1.2	Planning et évaluation	3
1.3	Qu'est-ce qu'un Cloud ?	3
1.4	Qu'est-ce que le Cloud Computing ?	3
1.4.1	Caractéristiques de base	4
1.5	Modèles de service	4
1.5.1	SaaS – Software as a Service	4
1.5.2	IaaS – Infrastructure as a Service	4
1.5.3	PaaS – Plateforme as a Service	4
1.5.4	Tableau comparatif IaaS / PaaS / SaaS	4
1.5.5	Autres modèles de services (XaaS)	4
1.6	Modèles de déploiement	5
1.6.1	Cloud Public	5
1.6.2	Cloud Privé	5
1.6.3	Cloud Communautaire	5
1.6.4	Cloud Hybride	5
1.7	Stratégie Cloud d'entreprise	5
1.8	Piliers technologiques (Enablers)	6
2	Virtualisation	6
2.1	Virtualisation et les clouds	6
2.2	Définitions	6
2.3	Types de virtualisation	6
2.3.1	Virtualisation complète	6
2.3.2	Para-Virtualisation	7
2.3.3	Virtualisation légère – Conteneurs (niveau OS/SE)	7
2.4	L'hyperviseur (VMM)	7
2.4.1	Types de VMM	7
2.5	Avantages et inconvénients	7
2.5.1	Virtualisation complète / Para-virtualisation (VMs)	7
2.5.2	Virtualisation légère (Conteneurs)	7
3	Virtualisation complète avec support matériel	8
3.1	Prérequis CPU (x86)	8
3.2	Support matériel : Intel VT-x / AMD-V	8
3.2.1	Fonctionnalités VT-x	8
3.3	Fonctionnement VMM / Systèmes invités	8
3.3.1	Rôle de la structure VMCS	8
3.3.2	EPT et TLB	9
3.4	KVM – Kernel-based Virtualization	9
3.5	QEMU – Quick EMUlator	9
3.6	Virtualisation imbriquée (Nested Virtualization)	10

4	Virtualisation de Réseaux	10
4.1	Objectifs	10
4.2	Interface TUN/TAP	11
4.3	Bridge (Niveau 2)	11
4.4	Interface macvtap	11
4.5	Connexion Host ↔ réseau externe	11
4.6	Réseaux virtuels avec routeurs	12
5	Sources bibliographiques	12

Introduction au Cloud

Contenu de l'UE CRV

- **Introduction au Cloud** : Définition, propriétés, modèles, services, technologies sous-jacentes
- **Virtualisation et réseaux virtuels** : Types et approches, mécanismes, supports système, création et gestion
- **Logiciels et interfaces Cloud** : CMS, IaC, Orchestration, AWS
- **Utilisation et services Cloud** : Pratiques DevOps, CI/CD, SRE, IoT

Objectif : Comprendre et maîtriser les technologies de création et d'exploitation de Cloud.

Planning et évaluation

- 10 × 2h de Cours (vendredi 8h30–10h30)
- 10 × 4h de TME avec rapports à soumettre
- Mini-projet d'UE avec AWS (après le Cours-TME 5)
- **Évaluation** : ER1 (30%), ER2 (30%), TME (30%), mini-projet AWS (10%)

ER1

Deux heures, tous documents autorisés. Interdits : téléphones, tablettes et ordinateurs.

Qu'est-ce qu'un Cloud ?

Définition : Le cloud est un ensemble de ressources informatiques accessibles à distance, partagées, facilement configurables, et rapidement approvisionnées.

Ressources d'un Cloud :

- Réseaux
- Serveurs
- Stockage
- Applications
- Services

Qu'est-ce que le Cloud Computing ?

Le cloud computing est un *modèle (approche)* de mise en place de ressources, d'applications et de services en utilisant un ou plusieurs clouds.

Il permet :

- Un accès au cloud : omniprésent, rapide, pratique, et **à la demande**
- Une utilisation avec un minimum d'effort de gestion ou d'interaction avec le fournisseur

Il est défini par ses **caractéristiques**, ses **modèles de service** et ses **modèles de déploiement**.

Caractéristiques de base

- Automatisme
- Accès hétérogène et standard au réseau du cloud
- Assemblage et partage des ressources au besoin → **Multitenancy** (locations multiples)
- Mesurabilité des services

Modèles de service

SaaS – Software as a Service

Le fournisseur fournit une application complète. Le client l'utilise sans maintenance.

Exemples : Online Microsoft Office, Google Drive, Amazon WorkDocs, Overleaf, Webmail, Dropbox.

IaaS – Infrastructure as a Service

Le fournisseur fournit uniquement le support nécessaire (VMs, réseaux, stockage). L'utilisateur installe l'OS, les bibliothèques, les applis et gère ses données. *“Infrastructure semi-vide à la demande”*.

Exemples : Scaleway Dedibox, OVHcloud VPS, AWS VPC, OCI Compute.

PaaS – Plateforme as a Service

Fournit une plateforme complète (OSs, Middlewares, APIs, environnements d'exécution, BD, serveurs Web). L'utilisateur se concentre sur le développement et le déploiement de ses services.

Exemples : AWS Chime SDK, Google Cloud App Engine, Red Hat OpenShift OpenJDK.

Tableau comparatif IaaS / PaaS / SaaS

IaaS	PaaS	SaaS
Interface (<i>user</i>)	Interface (<i>user</i>)	Interface (<i>fourni</i>)
Application (<i>user</i>)	Application (<i>user</i>)	Application (<i>fourni</i>)
Runtime (<i>user</i>)	Runtime (<i>fourni</i>)	Runtime (<i>fourni</i>)
Middleware (<i>user</i>)	Middleware (<i>fourni</i>)	Middleware (<i>fourni</i>)
Système d'exploitation (<i>user</i>)	OS (<i>fourni</i>)	OS (<i>fourni</i>)
Accès aux ressources (<i>user</i>)	Accès res. (<i>fourni</i>)	Accès res. (<i>fourni</i>)
Infrastructure Physique (<i>fourni</i>)	Infra. Phys. (<i>fourni</i>)	Infra. Phys. (<i>fourni</i>)

Autres modèles de services (XaaS)

- **CaaS** : Container as a Service – la couche “Conteneurs” est fournie par le provider
- **FaaS** : Function as a Service – l'utilisateur ne déploie que des fonctions
- **LBaaS** : Load Balancing as a Service
- **DBaaS** : Data Base as a Service
- **SECaaS** : Security as a Service

- MaaS : Monitoring as a Service
- BMaaS : Bare Metal as a Service

Modèles de déploiement

Cloud Public

Ouvert au grand public. Peut être détenu/géré par une entreprise, un universitaire ou un organisme gouvernemental. Il existe chez le fournisseur.

Exemples : OVHcloud, AWS, service de VMs du LIP6.

Cloud Privé

Réservé à une seule organisation (plusieurs consommateurs internes). Peut être géré par l'organisation elle-même, un tiers (managed cloud), ou les deux. Peut être on-premises ou off-premises.

Exemples : LIP6 Cloud, Comcast Cloud, WALMART e-commerce Cloud, Nubo Cloud (DGFIP).

Cloud Communautaire

Réservé à une communauté d'organisations partageant des préoccupations communes. Peut être géré par une organisation de la communauté ou un tiers.

Exemples : FIT IoT Lab, EdgeNet.

Cloud Hybride

Composition de deux ou plusieurs infrastructures cloud distinctes (souvent privé + public). Les clouds restent indépendants mais liés par une technologie standardisée permettant la portabilité des données/applications.

Exemple de technologie : **cloud bursting** (équilibre de charge entre cloud privé et public).

Exemples : Penn National Insurance Cloud (sur site + AWS), Kakao Cloud.

Stratégie Cloud d'entreprise

Le choix des modèles de déploiement et de service dépend de :

- Exigences réglementaires et politiques
- Localisation géographique des utilisateurs
- Montant initial d'investissement et ressources financières
- Modèle économique
- Prévisibilité de la demande
- Durée des projets impliquant le Cloud

Piliers technologiques (Enablers)

- Matériels puissants, à faible coût et faible consommation (serveurs, processeurs, baies, switchs, cartes réseaux...)
- **Virtualisation**
- Traitement parallèle et applications distribuées
- Logiciels Web
- Logiciels de gestion, d'orchestration et d'automatisation
- Modernisation des applications – approche **cloud-native**

Virtualisation

Virtualisation et les clouds

La virtualisation est une **importante technologie support** pour le modèle IaaS et autres XaaS. Elle permet au Cloud de supporter plusieurs et différents apps/systèmes/-réseaux sur différentes machines.

Dans le modèle IaaS :

Sans virtualisation	Avec virtualisation
Accès aux ressources	Virtualisation / Hyperviseur
Infrastructure Physique	Infrastructure Physique

Définitions

Virtualisation – Concept général

Création d'une interface **virtuelle** similaire à l'interface réelle visible d'un objet quelconque. L'interface virtuelle permet d'accéder aux ressources d'un autre objet ou du même objet.

Virtualisation de système

Création d'un logiciel/mécanisme permettant l'exécution d'un ou **plusieurs** systèmes pouvant être différents sur un autre système.

Le système virtualisé s'appelle : **machine virtuelle (VM)**, image virtuelle, système invité (*guest OS*).

Types de virtualisation

Virtualisation complète

Le logiciel de virtualisation assure à un autre système entier un **accès complet** aux ressources matérielles de la machine.

Supports :

- Émulation
- Translation binaire (*Binary Translation*)
- Support du CPU (*Hardware-assist*)

Exemples : VMWare Player, QEMU/KVM.

Para-Virtualisation

Le système **est modifié** pour interagir proprement avec le logiciel de virtualisation.

Exemples : XEN, ESX VSI.

Virtualisation légère – Conteneurs (niveau OS/SE)

Un ensemble de processus ont un accès **isolé** au système de fichiers, processeurs, mémoire, périphériques, etc. Certains objets du noyau peuvent être dupliqués.

Exemples : FreeBSD Jails, LXC.

L'hyperviseur (VMM)

Le composant principal du logiciel de virtualisation est appelé :

- **Superviseur (Hypervisor)** ou
- **Moniteur (Virtual Machine Monitor – VMM)**

Types de VMM

- **Type 1** : S'exécute *directement sur le matériel* (bare metal) pour capturer les instructions du système virtuel. → associé à la para-virtualisation.
- **Type 2** : S'exécute *au sein du système hôte réel*. Détecte les instructions du système virtuel et gère leurs accès aux ressources. → associé à la virtualisation complète.
- **Conteneurs (virtualisation niveau SE)** : Pas de moniteur. C'est le **noyau du système hôte** qui isole les processus et gère l'accès aux ressources.

Avantages et inconvénients

Virtualisation complète / Para-virtualisation (VMs)

Avantages :

- Système normal complet, illusion d'un équipement usuel
- Fournit le système d'exploitation exact requis par le client du Cloud

Inconvénients :

- Performance peut être limitée
- Nombre assez limité de VMs sur une seule machine du Cloud
- Dynamicité réduite
- Gaspillage de ressources du Cloud
- Partage des ressources Cloud moins flexible

Virtualisation légère (Conteneurs)

Avantages :

- Meilleure performance
- Nombre très grand de conteneurs sur la même machine
- Création et dimensionnement à la demande
- Dynamicité, rapidité et flexibilité

Inconvénients :

- Certains services sont plus difficiles à mettre en place dans des conteneurs
- Mise à jour continue du noyau du système hôte requise
- Nécessite plus de mécanismes de sécurisation

Virtualisation complète avec support matériel

Prérequis CPU (x86)

- **Instructions non-privilégiées** : s'exécutent dans l'espace utilisateur (Ring 3)
- **Instructions privilégiées** : uniquement en mode noyau (Ring 0)
- **Instructions sensibles** : sous-ensemble des instructions qui modifient le niveau de privilège du processeur, ou dont le comportement dépend du niveau de privilège

Si une instruction privilégiée est exécutée en mode non privilégié → **trap (interruption)** → l'hyperviseur prend la main et exécute l'action nécessaire.

Support matériel : Intel VT-x / AMD-V

- **Intel VT-x** : Virtualization Technology extension
- **AMD AMD-V** : AMD with Virtualization

Fonctionnalités VT-x

- Modes **Root** (VMM) et **Non-Root** (VM invitée)
- Flags CPU : **VMX** (Virtual Machine Extension), **EPT** (Extended Page Table)
- Structure : **VMCS** (Virtual Machine Control Structure) – une par vCPU
- Instructions VMX : VMXON, VMLAUNCH, VMRESUME, VMXOFF, VMCALL, VMCLEAR, INVEPT, INVVPID, VMFUNC, VMPTRLD, VMPTRST, VMREAD, VMWRITE

Fonctionnement VMM / Systèmes invités

1. Le CPU démarre en mode normal
2. VMXON → démarrage de la virtualisation en **mode root**
3. VMLAUNCH → démarrage d'une VM, passage en **mode non-root**
4. La VM exécute son propre code normalement...
5. ...jusqu'à une instruction non autorisée : **VM Exit** → passage en mode root
6. L'hyperviseur traite la raison du VM Exit
7. VMRESUME → retour en mode non-root, la VM reprend

Rôle de la structure VMCS

- Permet de savoir pourquoi le VM Exit s'est produit
- Distingue entre les VMs (une VMCS par vCPU)
- VMREAD/VMWRITE pour lire/écrire une VMCS
- VMPTRLD/VMPTRST pour activer une seule VMCS à la fois

EPT et TLB

- **EPT (Extended Page Table)** : implante la conversion d'adresse virtuelle en adresse physique à l'intérieur de la VM
- **TLB (Translation Lookaside Buffer)** : met en cache les mappages virtuels ↔ physiques pour accélérer les recherches dans les tables de pages

KVM – Kernel-based Virtualization

Virtualisation complète avec support matériel **implantée au niveau du noyau Linux**.

Avantages :

- Interaction plus simple entre matériel et hyperviseur
- Réutilisation des fonctions existantes du noyau (gestion mémoire, pilotage périphériques)

Modules noyau :

```
kvm.ko          # module de base
kvm_intel.ko     # module spécifique Intel
kvm_amd.ko       # module spécifique AMD
```

KVM exécute les **VMLAUNCH**, **VMRESUME**, met en place le **VMCS** et gère les **VM Exit**.

Important : KVM n'implante que la composante noyau. L'espace utilisateur est géré par → **QEMU**.

Vérifier les modules avec **lsmod** :

```
kvm_intel    380928    0
kvm          1146880    1 kvm_intel
```

Vérifier le support VT-x avec **lscpu** (chercher le flag **vmx**) :

```
Virtualisation : VT-x
```

QEMU – Quick EMUlator

- Utilitaire qui interagit avec KVM depuis l'**espace utilisateur**
- Crée un **processus par VM** et un **thread par vCPU**
- Dialogue avec KVM via **ioctl()** sur le périphérique **/dev/kvm**
- **Émule les Entrées/Sorties (I/O)** :
 1. VM exécute une instruction d'accès à un périphérique
 2. KVM prend la main et examine la raison
 3. KVM passe la main à QEMU
 4. QEMU émule le périphérique
 5. QEMU redonne le contrôle à KVM
 6. KVM exécute **VMRESUME**
- Effectue une **translation binaire** (via le module **TCG – Tiny Code Generator**) quand le système invité est destiné à une architecture autre que x86 (ex : MIPS, ARM) ou quand le support matériel est absent

Exemple de commande QEMU :

```
qemu -enable-kvm -name guest=myvm \  
-m 2048 \  
-smp cpus=2 \  
-drive file=/usr/Virtmachin/myvm.qcow2,format=qcow2 \  
-device usb-mouse,id=input2,bus=usb.0,port=3
```

Format QCOW (QEMU Copy-On-Write) : Permet la création d'*overlays* (clones légers basés sur une image disque existante) → concept important pour le Cloud Computing.

Virtualisation imbriquée (Nested Virtualization)

- Une VM avec hardware-assist **dans** une VM avec hardware-assist
- Un hyperviseur invité dans une VM sous un hyperviseur hôte

Usages :

- Permettre aux clients IaaS d'exécuter leur propre hyperviseur et VMs au-dessus du système loué
- Simplifier la migration d'un ensemble hyperviseur + VMs
- Expérimentation de nouvelles technologies
- Augmenter la sécurité de l'hyperviseur
- Partage hiérarchique des ressources virtualisées

Les processeurs Intel et AMD supportent la virtualisation imbriquée. Les VM Exit sont toujours interceptées en premier par l'**hyperviseur hôte (L0)**.

Activation dans KVM :

```
cat /sys/module/kvm_intel/parameters/nested  
# Y  
  
# ou dans /etc/modprobe.d/kvm.conf :  
options kvm_intel nested=1  
  
# ou directement :  
modprobe kvm_intel nested=1
```

Virtualisation de Réseaux

Objectifs

- **Connecter les VMs entre elles :** réseaux de VMs similaires aux réseaux classiques (commutateurs, routeurs, liens, interfaces)
- **Connecter les VMs au réseau du système hôte**
- **Connecter les VMs au réseau physique externe (Internet)** → Networking service dans les Clouds

Interface TUN/TAP

Pour connecter le système invité au système hôte, on transmet les trames Ethernet dans les deux sens entre les deux noyaux. On utilise les interfaces de type **TUN/TAP**, généralement en mode **TAP**.

Création :

```
ifconfig tap0 create          # FreeBSD
ip tuntap add tap0 mode tap   # Linux
```

Principe : Permet l'échange de trames Ethernet entre un processus et le système via des appels classiques `read()` et `write()` :

- `write()` : processus → interface → système
- `read()` : système → interface → processus

Interface classique	vs.	Interface TAP
Carte réseau (matérielle)		Processus

Bridge (Niveau 2)

Le module noyau **bridge** (Linux) ou **if_bridge** (FreeBSD) émule un switch ou un pont connectant plusieurs interfaces.

```
ifconfig bridge create        # FreeBSD
ip link add bridge0 type bridge # Linux
```

Architecture typique Guest → Host :

- **Guest OS** : NIC driver
- **Logiciel de virtualisation** : émulateur NIC
- **Host** : interface TAP → bridge → NIC hardware

Interface macvtap

Alternative à tap+bridge, avec plusieurs modes :

Mode	Comportement
Pont	Équivalent à tap + bridge. Le switch externe n'est pas impliqué.
VEPA	VM vers VM via le switch externe (hairpin/RR requis).
Privé	Communications VM à VM de niveau 2 interdites.
Passthrough	Une seule VM connectée directement à l'interface physique.

Connexion Host ↔ réseau externe

Plusieurs approches depuis le bridge :

- Connexion directe à la pile protocolaire du système hôte
- Via **NAT** (Network Address Translation)
- Via **routage**

Réseaux virtuels avec routeurs

Il est possible de créer des réseaux multi-segments en utilisant une VM comme routeur :

```
sysctl net.inet.ip.forwarding=1      # FreeBSD
sysctl net.ipv4.conf.eth0.forwarding=1 # Linux
# + processus de routage ou routage statique
```

Architecture : VM1 ↔ bridge1 ↔ VM_routeur ↔ bridge2 ↔ VM2

Sources bibliographiques

- Dan C. Marinescu. *Cloud Computing Theory and Practice*. MK Morgan Kaufmann. 3^e éd. 2022–2023. ISBN 978-0-323-85277-7.
- R. Buyya, C. Vecchiola and S.T. Selvi. *Mastering Cloud Computing Foundations and Applications Programming*. ELSEVIER-MK. 2013. ISBN 978-0-12-411454-8.
- OpenStack Documentation : <https://docs.openstack.org/>
- QEMU documentation : <https://www.qemu.org/documentation/>
- Kernel Virtual Machine : <https://linux-kvm.org/>
- VMWare Technical Papers : <https://www.vmware.com/techpapers.html>
- Intel Developer Manual Volume 3C : <https://www.intel.fr/content/www/fr/fr/content-details/812396>